

GBNet

XGBoost and LightGBM PyTorch Modules

<https://joss.theoj.org/papers/10.21105/joss.08047>



Michael Horrell, PhD

Director AI & Data at AlixPartners



mthorrell



@horrellmt



@horrellmt.bsky.social

Overview

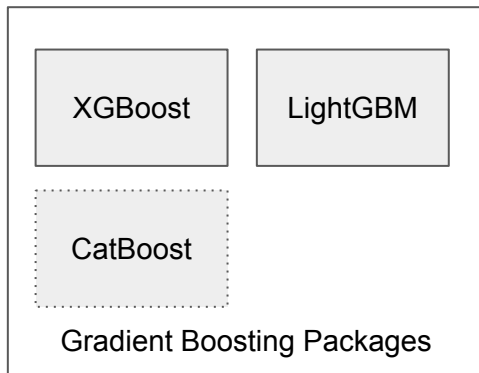
What is GBNNet?

GBNet provides PyTorch Module components that call XGBoost and/or LightGBM under the hood so you can build complex models/architectures that use these Gradient Boosting (GBDT) implementations and train everything end-to-end

Who is this presentation for?

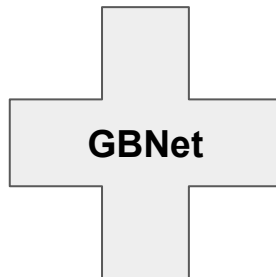
- Users of XGBoost or LightGBM
- Users of PyTorch
- Anyone interested in statistical modeling/model architectures with some non-parametric components

GBNet uses PyTorch to provide GBDT packages with gradients for model fitting

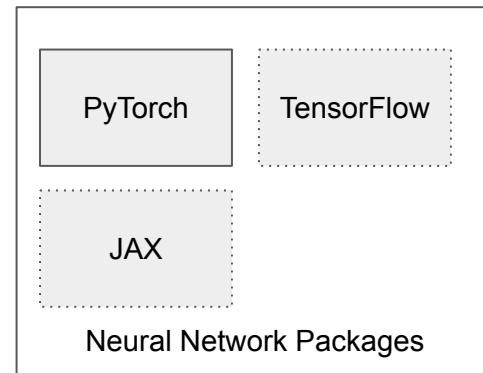


Gradient Boosting Packages train using three main inputs:

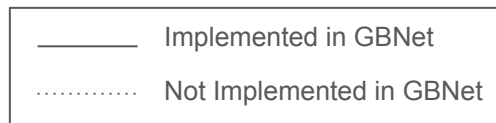
- X inputs
- G loss gradients
- H loss Hessians



GBNet leverages Neural Network auto-differentiation so you can use your favorite Gradient Boosting package as a standard component of a Neural Network

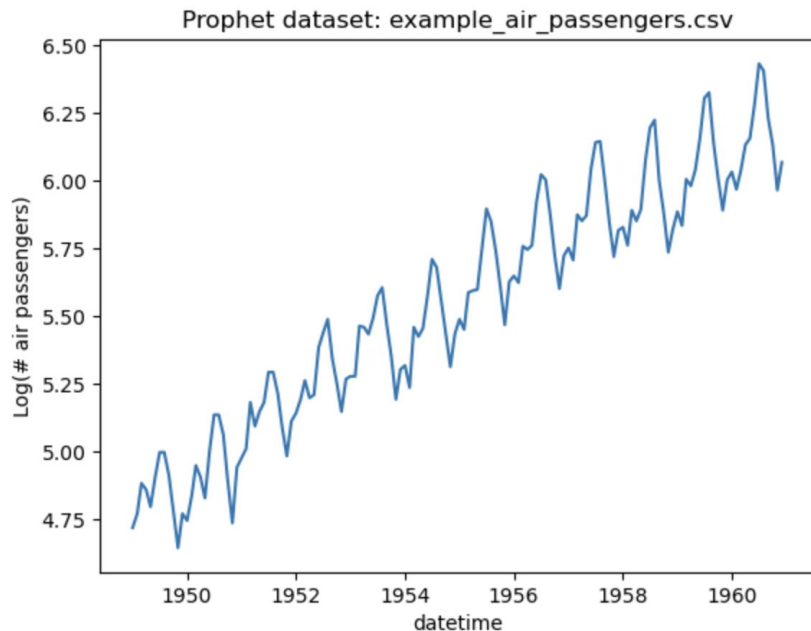


Neural network packages calculate gradients and Hessians with auto-differentiation



Forecasting example: Use GBNet to solve a broader set of problems using XGBoost and/or LightGBM

Your data looks like this:



You want to fit the following model, using XGBoost for the seasonal component

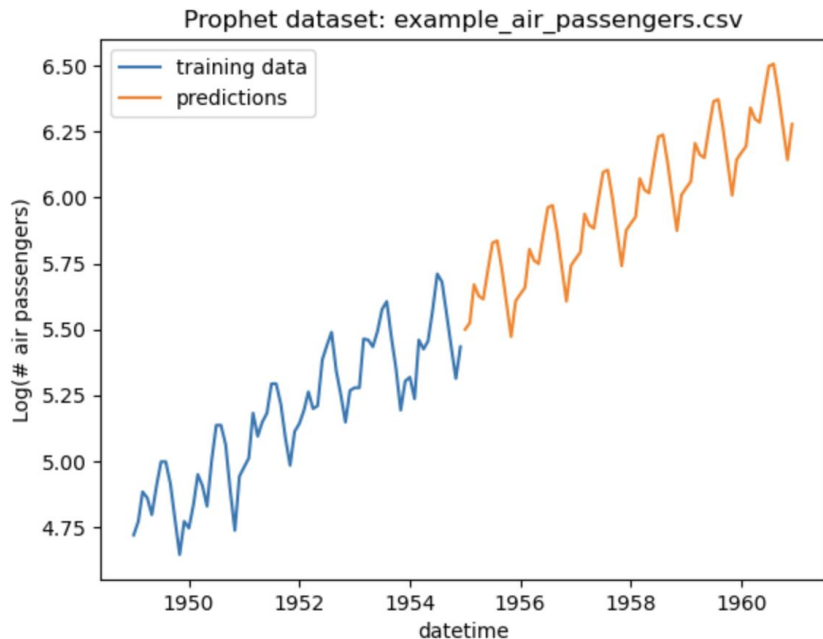
$$F(t) = t\beta + \text{Seasonality}(t)$$

Options:

1. Backfitting
2. Some kind of two step fitting of β and then also transforming the response
3. **GBNet!**

Forecasting example: A few lines using GBNets XGBModule yields an expressive and easily fit model

Train on first 50%; predict on last 50%



```
import numpy as np
import pandas as pd
import torch

from gbnets.xgbmodule import XGBModule
```

```
class BasicForecast(torch.nn.Module):
    def __init__(self, n):
        """ 3 components: trend, seasonality, batchnorm for trend
        """
        super(BasicForecast, self).__init__()
        self.season = XGBModule(n, 1, 1)
        self.bn = torch.nn.LazyBatchNorm1d()
        self.linear = torch.nn.Linear(1, 1)

    def forward(self, pd_datetime_col):
        # Convert to usable components
        numeric_years = np.array(
            pd.to_numeric(pd_datetime_col).to_frame()
            / (365.25 * 24 * 60 * 60 * 1_000_000_000)
        )
        month = pd_datetime_col.dt.month

        # result = linear + season
        season = self.season(month.to_frame())
        linear = self.linear(self.bn(torch.Tensor(numeric_years)))
        return linear + season

    def gb_step(self):
        self.season.gb_step()
```

Calculation is straightforward and is only about 20 lines fully written out

Batchnorm needed for PyTorch component

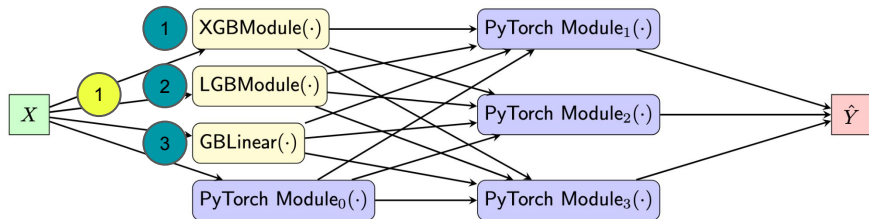
Seasonality is a function of month; trend operates on a numeric datetime

GBNet Modules can be used like any other PyTorch Modules but there are some caveats

Features:

GBNet has three PyTorch Modules that can be connected with other PyTorch Modules

- 1 XGBModule integrates XGBoost
- 2 LGBModule integrates LightGBM
- 3 GBLinear fits a linear layer via boosting (ie uses first and second order information)



Caveats:

- 1 Decision trees have zeros as gradients
- 2 There is some additional boilerplate code needed for fitting
- 3 Mini-batching requires special consideration

```
class GBPlus(torch.nn.Module):
    def __init__(self, n, input_dim, intermediate_dim, output_dim):
        super(GBPlus, self).__init__()
        self.xgb = xgbmodule.XGBModule(n, input_dim, intermediate_dim, {'eta': 0.1})
        self.lgb = lgbmodule.LGBModule(n, input_dim, intermediate_dim, {'eta': 0.1})
        self.linear = torch.nn.Linear(intermediate_dim, output_dim)

    def forward(self, input_array):
        xpreds = self.xgb(input_array)
        lpreds = self.lgb(input_array)
        preds = self.linear(xpreds + lpreds)
        return preds

    def gb_step(self):
        self.xgb.gb_step()
        self.lgb.gb_step()
```




Live Demo Time!

https://github.com/mthorrell/gbnet/blob/scipy2025/examples/SciPy_Live_Demo.ipynb

gbnet.models have end-to-end applications with sklearn-like interfaces

Application	Location	Model Description
Forecasting	gbnet.models.forecasting	“Curve fitting” type of Forecasting supporting trend changepoints and prediction uncertainty estimates. Similar to Meta’s Prophet algorithm. - Changepoints use GBDT Modules: $\text{Trend}(t) = \beta_0 + \beta_1 t + \sum_i^{\text{\#changepoints}} \text{GBDT}(i) \mathbb{I}_{(t \geq c_i)}(t - c_i)$ - Out-of-the-box, has better RMSE compared to Meta’s Prophet on benchmarks as seen in this notebook.
Ordinal Regression	gbnet.models.ordinal_regression	- Models the cumulative logit exactly: $P(y \leq i X) = \sigma(\theta_i - \text{GBDT}(X))$ - Has improved performance on ordinal datasets over other standard approaches (Classification, Huber, MAE). Notebook here.

Future Work: Models/Applications

- Survival analysis and analysis of Poisson Processes

$$y \sim \text{Pois} \left(\int_a^b \text{GBDT}(X(t); t) dt \right)$$

- Mixture of Experts; general mixture modeling

$$\hat{y} = w_1 \text{GBDT}_1(X) + w_2 \text{GBDT}_2(X)$$
$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = \text{GBDT}_3(X)$$

- More work with embeddings
 - Recommender systems
 - Variational embeddings
 - Contrastive training

$$M = \text{GBDT}_{rows} \text{GBDT}_{columns}^T$$

Future Work: Codebase

Enhancements

- Hessian calculation contains a python loop, can we remove it?
- Helpers to support mini-batch training
- End-to-end production functionality
- Use of PyTorch autograd to avoid warnings?

Hardening

- GBNet does some gymnastics to update XGBoost and LightGBM by single boost rounds. Is there a better way to do single tree updates for these packages?

Questions



- Questions?
- <https://github.com/mthorrell/gbnet>
- X @horrellmt
- 🦋 @horrellmt.bsky.social